

Table of contents

List of Figures	2
List of Tables	3
List of Abbreviations	4
1. Overview of internship	1
1.1. Introduction	1
1.2. Artix-7	1
2. Methods	2
2.1. Cooley-Tukey: Radix-2 DIT	2
2.1.1. Implementation in SystemVerilog	4
2.1.2. Schematic	5
2.1.3. Simulation	6
2.1.4. Implementation results	6
2.2. Radix-2 Multi Path Delay Commutator (R2MDC)	7
2.2.1. Schematic	8
2.2.2. Simulation result	9
2.2.3. Implementation result	10
2.2.3.1. Memory, Nets & DSP	10
2.3. Radix-2 Single Path Delay Commutator (R2SDC)	10
2.3.1. Schematic	11
2.3.2. Simulation result	13
2.3.3. Implementation results	14
2.3.3.1. Memory, Nets & DSP	14

List of Figures

Figure 2.1	Steps involved in Cooley-Tukey algorithm	2
Figure 2.2	Signal flow graph of 8-point radix-2 DIT-FFT	3
Figure 2.3	Block diagram of Radix-2 DIT FFT implementation	5
Figure 2.4	Working of FSM in Radix-2 DIT FFT	6
Figure 2.5	Timing diagram of output of 4 point Radix-2 DIT-FFT	6
Figure 2.6	Signal flow graph of 4-point radix-2 DIF-FFT	7
Figure 2.7	Steps in R2MDC algorithm with $N = 4$ case	8
Figure 2.8	Schematic of R2MDC implementation	9
Figure 2.9	Inside a single R2MDC Stage. All modules with wedge is clocked	9
Figure 2.10	Simulation result for 4 point complex test signal	9
Figure 2.11	R2SDC algorithm	12
Figure 2.12	Schematic of R2SDC implementation	13
Figure 2.13	Inside a single R2SDC Stage. All modules with wedge is clocked	13
Figure 2.14	Simulation result for 4 point complex test signal	13

List of Tables

List of Abbreviations

DFT:	Discrete Fourier Transform
DIF:	Decimation in Frequency
DIT:	Decimation in Time
DSP:	Digital Signal Processing
FFT:	Fast Fourier Transform
FT:	Fourier Transform
WNS:	Worst Negative Slack

Chapter 1:

Overview of internship

1.1 Introduction

In Digital Signal Processing (DSP), the need to express a time domain signal to frequency domain often comes in various settings. For this Fourier Transform (FT) is widely employed. However FT is computationally expensive. In past century various algorithms have been proposed to compute FT of signals with limited compute power and memory. Even to this date, various forms of FT algorithms have been proposed. These set are now commonly called Fast Fourier Transform (FFT) algorithms.

My internship is involved in implementing few of these algorithms using SystemVerilog suitable for FPGA. The architecture and operation of these algorithms are detailed in the following chapter, as well as the hardware requirements and some performance aspects are also discussed along with them. In the last chapter, a comparative analysis of all algorithms are outlayed.

For analysis purpose Artix-7 FPGA is used in all the implementation. Operating frequency of 100 MHz is targeted in all algorithms. However as we will some algorithm cannot achieve this while others can go beyond this point.

1.2 Artix-7

Chapter 2:

Methods

2.1 Cooley-Tukey: Radix-2 DIT

Cooley-Tukey algorithm re-expresses the Discrete Fourier Transform (DFT) of an arbitrary composite size $N = N_1 N_2$ in terms of N_1 smaller DFTs of sizes N_2 , recursively, to reduce the computation time to $\mathcal{O}(N \log N)$ for highly composite N . Because of the algorithm's importance, specific variants and implementation styles have become known by their own names, such as Radix-2, Radix-4, mixed-radix, etc. Because the Cooley-Tukey algorithm breaks the DFT into smaller DFTs, it can be combined arbitrarily with any other algorithm for the DFT.

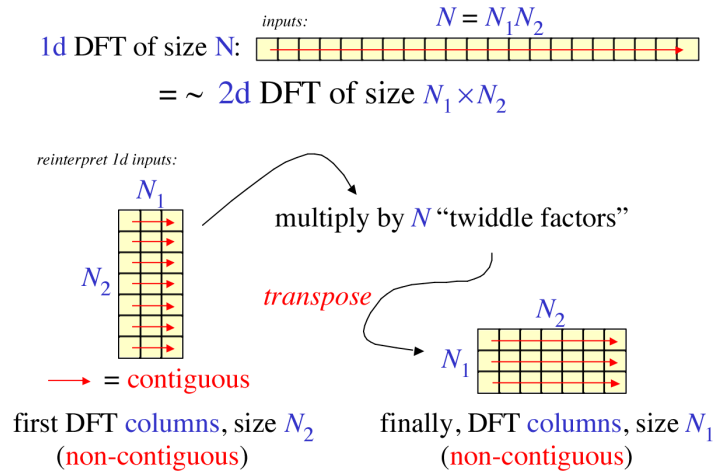


Figure 2.1: Steps involved in Cooley-Tukey algorithm

A radix-2 Decimation in Time (DIT) FFT is the simplest and most common form of the Cooley-Tukey algorithm. Radix-2 DIT divides a DFT of size N into two interleaved DFTs of size $N/2$ with each recursive stage. Hence the overall time complexity of algorithm is $\mathcal{O}(N \log N)$. The following is the derivation of radix-2 DIT:

DFT of sequence x_n is defined by:

$$X_k = \sum_{n=0}^{N-1} x_n \exp\left(-j \frac{2\pi}{N} nk\right)$$

The radix-2 DIT algorithm rearranges the DFT of the function x_n into two parts: a sum over the even-numbered indices $n = 2m$ and a sum over the odd-numbered indices $n = 2m + 1$:

Implementation of FFT

$$\begin{aligned}
 X_k &= \sum_{m=0}^{N/2-1} x_{2m} \exp\left(-j\frac{2\pi}{N}(2m)k\right) + \sum_{m=0}^{N/2-1} x_{2m+1} \exp\left(-j\frac{2\pi}{N}(2m+1)k\right) \\
 &= \underbrace{\sum_{m=0}^{N/2-1} x_{2m} \exp\left(-j\frac{2\pi}{N}mk\right)}_{\text{DFT of even-indexed part of } x_n} + \exp\left(-j\frac{2\pi}{N}k\right) \underbrace{\sum_{m=0}^{N/2-1} x_{2m+1} \exp\left(-j\frac{2\pi}{N}mk\right)}_{\text{DFT of odd-indexed part of } x_n} \\
 &= E_k + \exp\left(-j\frac{2\pi}{N}k\right) O_k
 \end{aligned}$$

Using the periodicity of the complex exponential, $X_{k+\frac{N}{2}}$ is also obtained from E_k and O_k as:

$$X_{k+\frac{N}{2}} = E_k - \exp\left(-j\frac{2\pi}{N}k\right) O_k$$

We can write X_k and $X_{k+\frac{N}{2}}$ as:

$$X_k = E_k + \exp\left(-j\frac{2\pi}{N}k\right) O_k$$

$$X_{k+\frac{N}{2}} = E_k - \exp\left(-j\frac{2\pi}{N}k\right) O_k$$

This result, expressing the DFT of length N recursively in terms of two DFTs of size $N/2$, is the core of the radix-2 DIT FFT. The algorithm gains its speed by re-using the results of intermediate computations to compute multiple DFT outputs. The final outputs are obtained by combination of E_k , O_k and $\exp(-j2\pi k/N)$ (called “twiddle factor” or phase rotation factor) which is simply a size-2 DFT (also called “Butterfly”). The operation can be expressed as a signal flow graph shown in Figure 2.2

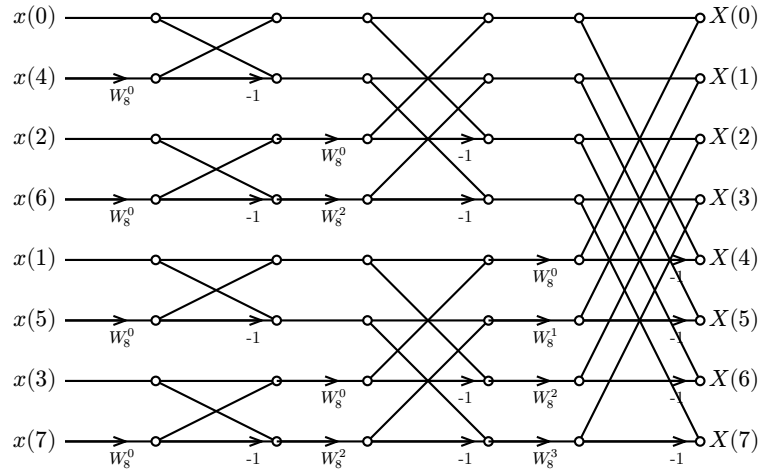


Figure 2.2: Signal flow graph of 8-point radix-2 DIT-FFT

In the software, assuming we have array of input x , the computation begins with reordering the array by swapping element at index i with element at an index which is bit reverse of i . Then subsequently performing radix-2 recursively.

2.1.1 Implementation in SystemVerilog

The SystemVerilog was chosen because of many features which improves ease of development. The implementation accpets a *fixed point* complex number (given through 2 separate buses for real and imaginary part) and returns computed DFT numbers through another 2 separate buses. The operation described below:

1. A reset pulse is applied
2. Start signal is issued which starts a counter inside the module.
3. `input_valid` signal is applied and input is passed in each positive edge of clock.
4. After N clock cycle, when all input are be given, `input_valid` is reset and module starts computing
5. After $N \log_2(N)/2$ clock cycles the output will be available
6. To read output, output read enable is raised and read address is passed. In the next clock cycle the output will be obtained.

The whole process takes $2N + N \log_2(N)/2 + 1$ clock cycles from passing first input to getting last output: $2N$ cycles for applying input and getting output and $N \log_2(N)/2 + 1$ cycles for computation. The module consists of

- 1 butterfly module
- 1 internal memory of size N
- 1 counter
- 1 address generator
- 1 bit reversing module
- 1 twiddle factor generator
- a control logic (FSM)

In theory area wise this should be most area efficient design. The design is also fully parametric – the number of points and data width can be changed. The twiddle factor is generated using a python script. Design is limited in terms of throughput and latency and is not useful for high performance use cases.

2.1.2 Schematic

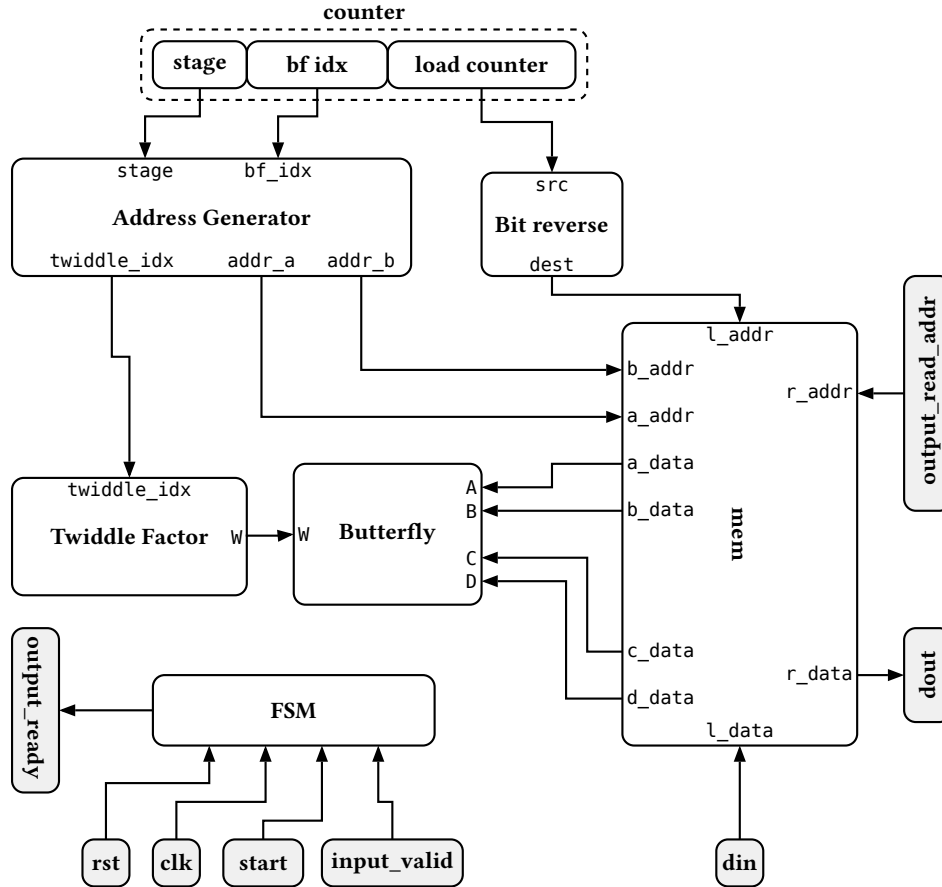


Figure 2.3: Block diagram of Radix-2 DIT FFT implementation

When a start signal is sent, the internal counter starts running. Based on the value of counter address to the addresses of relevant data – inputs to butterfly and twiddle factor – is generated by address generator. A portion of the counter is bit reversed to produce an address that will be used to store the input. The computing modules and FSM all also run in parallel. The FSM has 4 states:

- IDLE (not doing any processing)
- LOAD (loading input to internal memory)
- COMPUTE (computing the results and storing it in the memory)
- DONE: All computations are done and ready for next cycle.

The operation of FSM is summarized in Figure 2.4.

Implementation of FFT

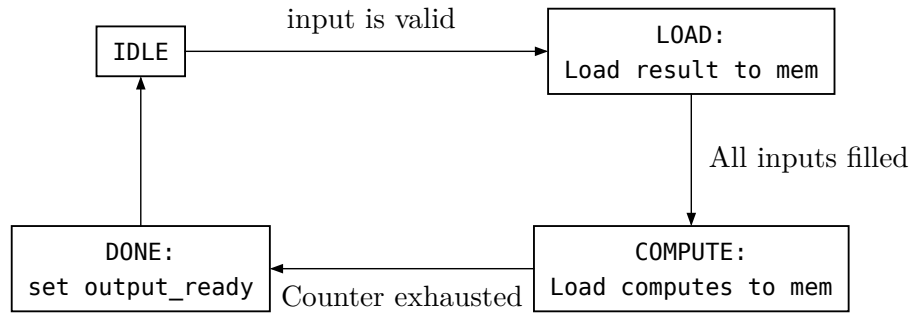


Figure 2.4: Working of FSM in Radix-2 DIT FFT

After the output is ready, the can be read by raising `output_read_en` and passing the address to be read in `output_read_addr`.

2.1.3 Simulation

Here 4 point DFT is computed in which input is real valued ramp signal. In the Figure 2.5, the output is observed at `dout_re` and `dout_im`. The results were verified using *GNU Octave*.

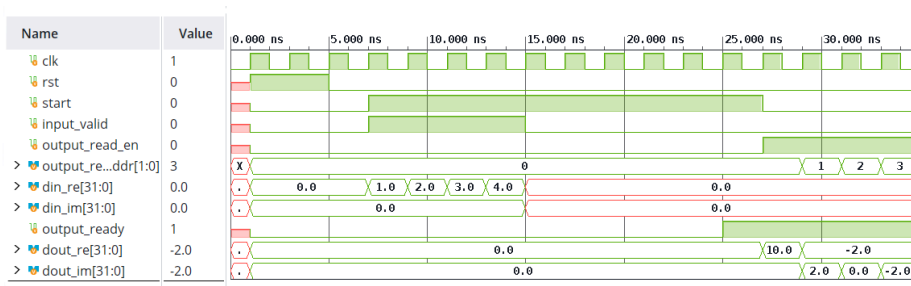


Figure 2.5: Timing diagram of output of 4 point Radix-2 DIT-FFT

2.1.4 Implementation results

The **Artix-7 (part no: xc7a100tcs9324-1)** FPGA was chosen for synthesis. Adjusting for slack, the current design works at maximum of 80.64 MHz. But in vivado the default implementation step still produces a layout which has Worst Negative Slack (WNS) of -0.2ns . Using the TCL command `phys_opt_design -directive Explore` brought it to $+0.095\text{ns}$.

The implementation is by no means efficient for practical use, but serves as a base upon which other algorithms can be implemented and evaluated.

table: for 16 Bit FIXP=10, N=8 obtained by: `report_utilization -hierarchical`

Instance	Module	Total LUTs	Logic LUTs	LUTRAMs	SRLs	FFs	RAMB36	RAMB18	DSP Blocks
fft_radix2	(top)	1106	1106	0	0	306	0	0	4
(fft_radix2)	(top)	116	116	0	0	306	0	0	0
bs1	butterfly_structure	990	990	0	0	0	0	0	4

Implementation of FFT

report_route_status
Design Route Status

	:	# nets :
-----	:	-----
# of logical nets.....	:	1723 :
# of nets not needing routing.....	:	461 :
# of internally routed nets.....	:	461 :
# of routable nets.....	:	1262 :
# of fully routed nets.....	:	1262 :
# of nets with routing errors.....	:	0 :
-----	:	-----

2.2 Radix-2 Multi Path Delay Commutator (R2MDC)

This algorithm is based on the Decimation in Frequency (DIF) method. In DIF, the inputs are applied in order, while output obtained will be in bit-reversed form. This allows the input to be fed sequentially. The DIF still gives similar signal flow graph as that of DIT. However the final output has particular pattern that can be used to transform this parallel structure to a pipelined multistaged architecture.

The usual structure of DIF in the case where $N = 4$ is shown in Figure 2.6. Notice that in stage 0, k th and $(k + \frac{N}{2})$ th inputs are required (x_0, x_2 and x_1, x_3). If the data is fed sequentially then one can store first 2 data into a shift register and pop out the value afterwards. This gives the input required for butterfly in the next stage. So the corresponding outputs in each stages are: x'_0, x'_2 and x'_1, x'_3 . But the next stage needs pairs x'_0, x'_1 and x'_2, x'_3 . So twiddle multiplied output of this stage needs to be stored for 1 cycle. Which will satisfy condition for next stage.

In general at stage $s(= 0, 1, \dots, S$ where $S = \lceil \log_2(N) \rceil =$ number of stages) needs shift register of size 2^{S-s} in upper input arm of butterfly and 2^{S-s-1} shift register after the twiddle path. For last stage, multipliers and commutators are not needed. For higher point FFT, design can be scaled easily by cascading more and more R2MDC stages.

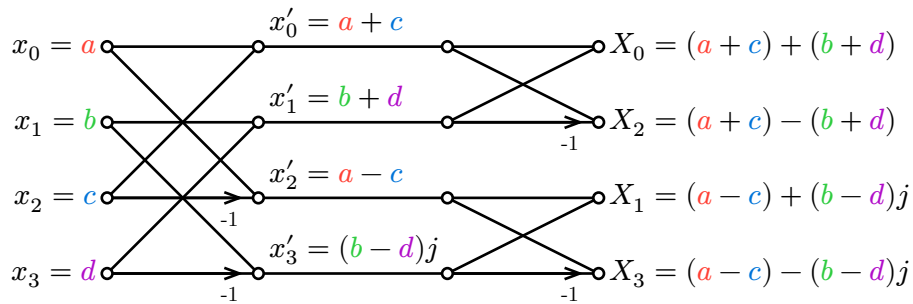


Figure 2.6: Signal flow graph of 4-point radix-2 DIF-FFT

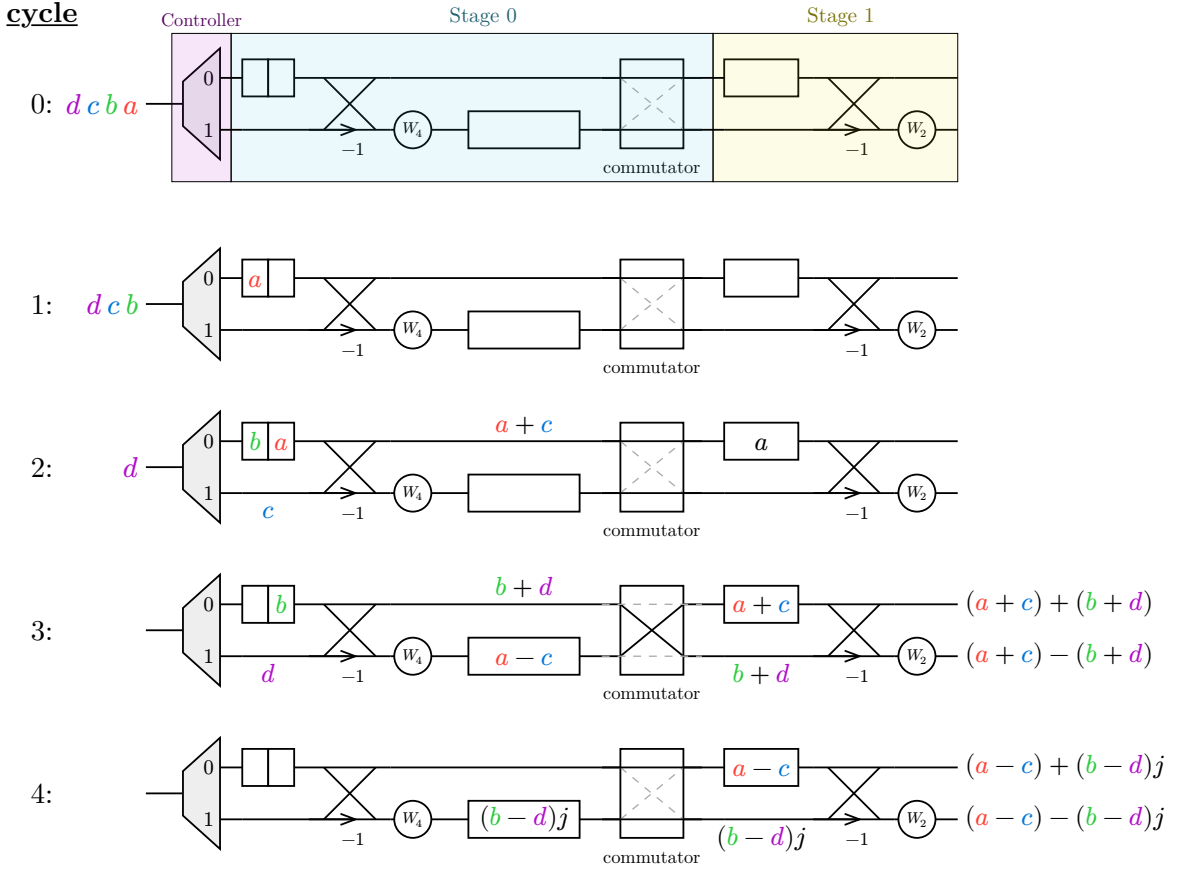


Figure 2.7: Steps in R2MDC algorithm with $N = 4$ case

2.2.1 Schematic

The design is fully parametric with parameters:

- **STAGES**: number of stages. (hence total number of input points = 2^{STAGES})
- **DW**: Data width (default: 16 bits)
- **FIXP_Q**: Number of bits for decimal part (default: 10)

The design consists of a controller block that sets the starts the global counter and produces control signals. The outputs **out0** and **out1** are passed to cascade of R2MDC Stage blocks as shown in Figure 2.8. Each R2MDC stage is a pipelined design consisting of a butterfly unit, twiddle ROM and commutator. Butterfly here only computes sum and difference of incoming inputs, the multiplication with twiddle factor is done using a pipelined multiplier. The required twiddle factors are computed during synthesis and are stored in a LUT. The control signals are also delayed by the same amount as that of data lines.

In each stage the size of delay element reduces by half. No delays, multipliers and commutators are required in final stage. The final output is obtained at **out0** and **out1** corresponding to bit reversed indexed $X[k]$ and $X[k + N/2]$. The control signals **valid** and **counter** are synchronously passed from one stage to another, using delay. As shown in Figure 2.9, 2 time

Implementation of FFT

unit delay is used for pipelining: one for splitting addition and multiplication in butterfly and another one for pipelining complex multiplication in multiplier.

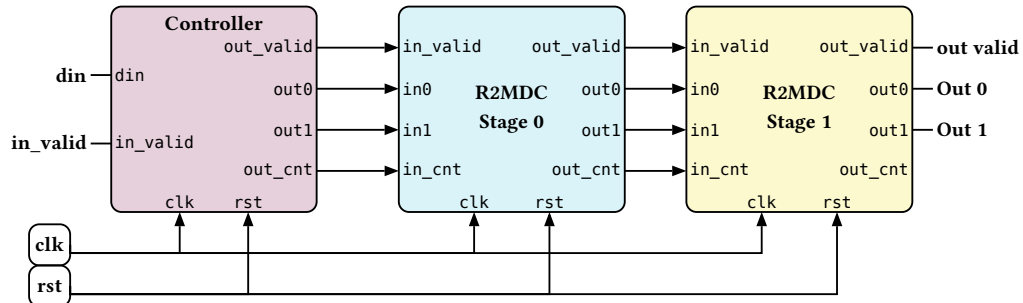


Figure 2.8: Schematic of R2MDC implementation

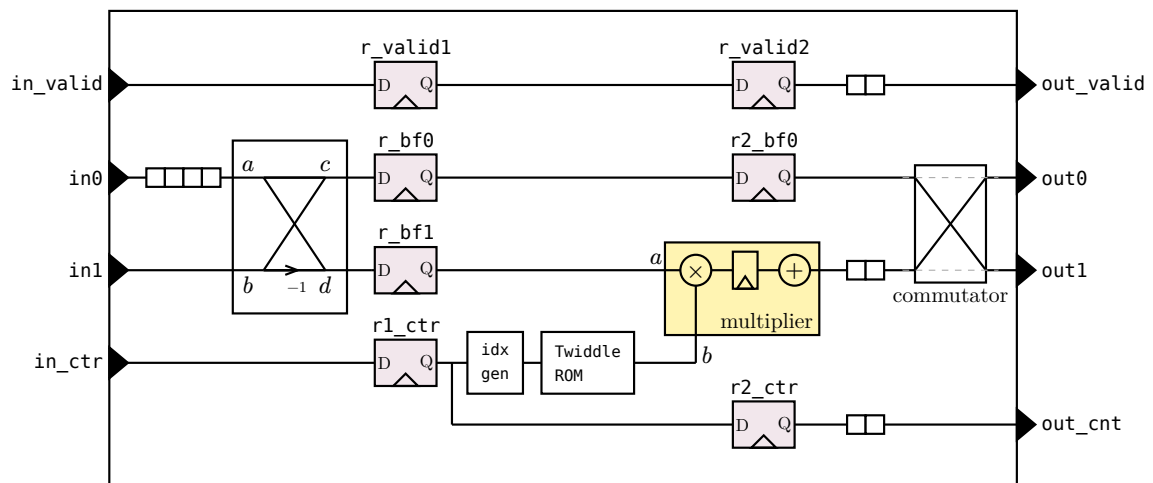


Figure 2.9: Inside a single R2MDC Stage. All modules with wedge is clocked

2.2.2 Simulation result

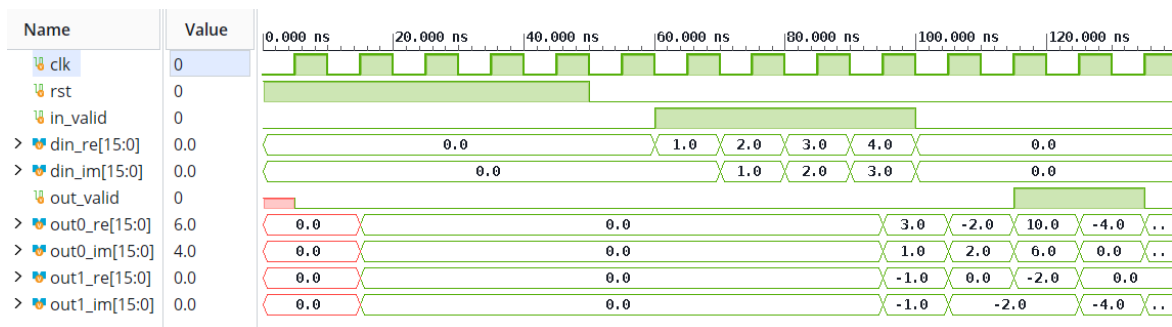


Figure 2.10: Simulation result for 4 point complex test signal

Implementation of FFT

2 outputs are obtained at same clock cycle. Hence it takes $N + N/2$ clock cycles to get all outputs after 1st input has been inserted.

2.2.3 Implementation result

[

2.2.3.1. Memory, Nets & DSP

N	Total LUT	LUT/N	Logic LUT	LUT RAM	SRL	FF	FF/N	RAMB 36	RAMB 18	DSP Block	DSP Block/N
8	279	34.8	246	0	33	404	50.5	0	0	8	1
16	419	26.1	352	0	67	645	40.3	0	0	12	0.75
32	613	19.1	479	0	134	862	26.9	0	0	16	0.50
64	809	12.6	606	0	203	1091	17.0	0	0	20	0.312
128	1049	8.1	744	0	305	1340	10.4	0	0	24	0.187
256	1315	5.1	804	0	511	1627	6.3	0	8	28	0.109
512	1779	3.4	923	0	856	1967	3.8	0	10	32	0.062
1024	2831	2.7	1040	0	1791	2217	2.1	0	12	36	0.035
2048	4789	2.3	1166	0	3623	3214	1.5	0	12	40	0.019
4096	8156	1.9	1269	0	6887	4455	1.0	14	0	44	0.010
8192	15189	1.8	1390	0	13799	6746	0.8	16	16	48	0.005

N	All Nets	Logical Net	All Nets/N	Logical Net/N	Power (mW) *	WNS (ns)	$F_{\max}$
8	3531	1357	441.3	169.6	176	3.129	145.53
16	5139	1982	321.1	123.8	187	2.709	137.15
32	6811	2657	212.8	83.0	197	2.655	136.14
64	8511	3347	132.9	52.2	209	2.403	131.63
128	10269	4092	80.2	31.9	221	1.755	121.28
256	12541	5030	48.9	19.6	250	1.745	121.13
512	14734	6192	28.7	12.0	264	1.589	118.89
1024	17278	7695	16.8	7.5	293	1.459	117.08
2048	21627	10929	10.5	5.3	332	1.257	114.37
4096	27581	15826	6.7	3.8	419	1.430	116.68
8192	38364	25434	4.6	3.1	571	1.569	118.60

] * - on chip power (Confidence Level: Medium)

2.3 Radix-2 Single Path Delay Commutator (R2SDC)

Single path Delay Commutator is a different pipelined implementation of DIF algorithm. Unlike MDC, SDC produces single output per cycle and uses fewer number of shift registers,

Implementation of FFT

at the expense of higher latency and higher complexity of stage controllers. It works as following:

1. In first $N/2$ clock cycles data coming is shifted to shift registers
2. In coming cycles, $x[n]$ and $x[n + N/2]$ datas are available. The butterfly computes the sum and difference, in which the difference is shifted back to the same shift register and sum is passed to twiddle factor multiplier.
3. After all inputs are passed (cycles $> N$) the shift register is completely full of differences. It shifts it out to twiddle factor directly.

2.3.1 Schematic

The design is fully parametric with parameters:

- **STAGES:** number of stages. (hence total number of input points = 2^{STAGES})
- **DW:** Data width (default: 16 bits)
- **FIXP_Q:** Number of bits for decimal part (default: 10)

Implementation of FFT

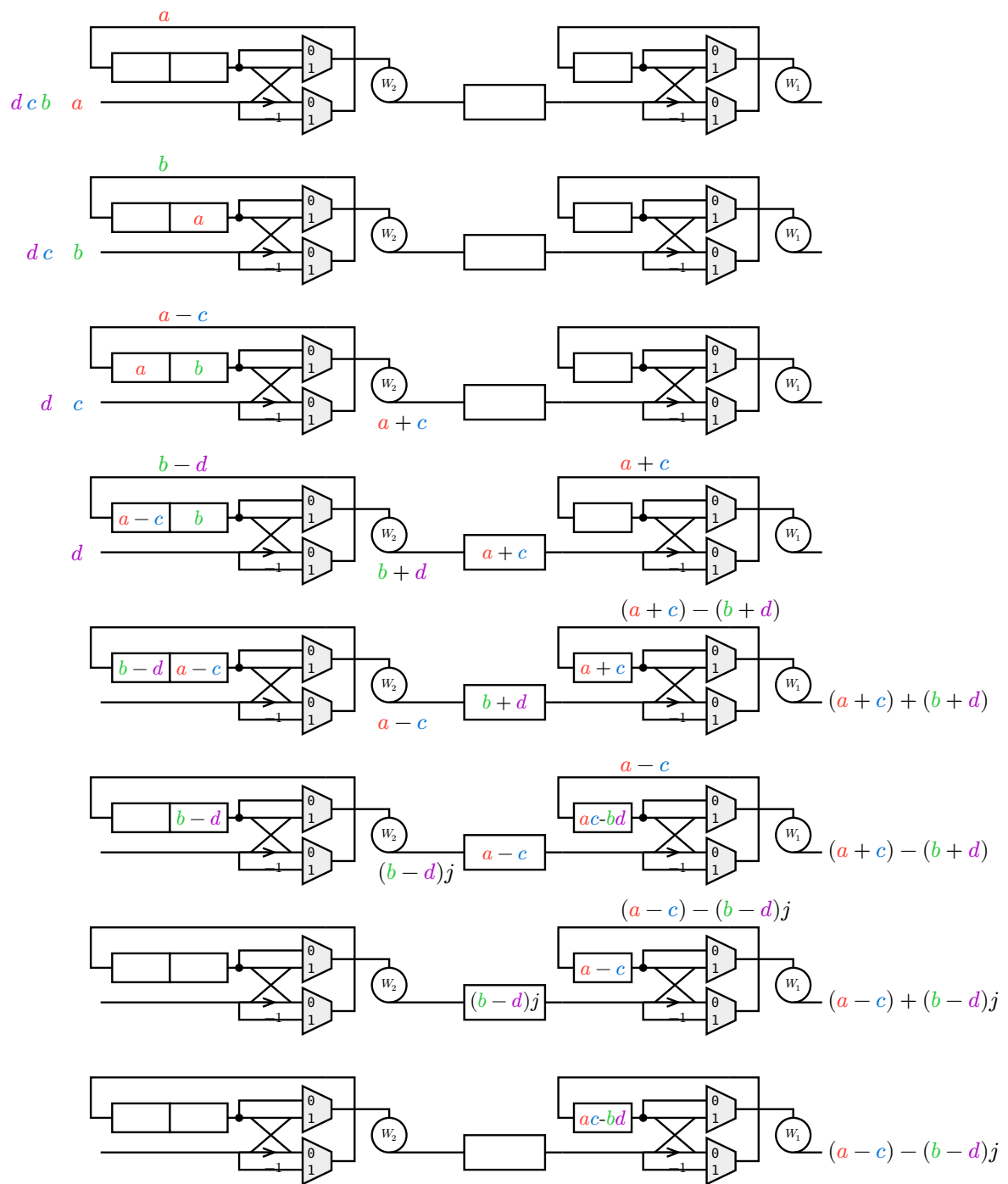


Figure 2.11: R2SDC algorithm

Implementation of FFT

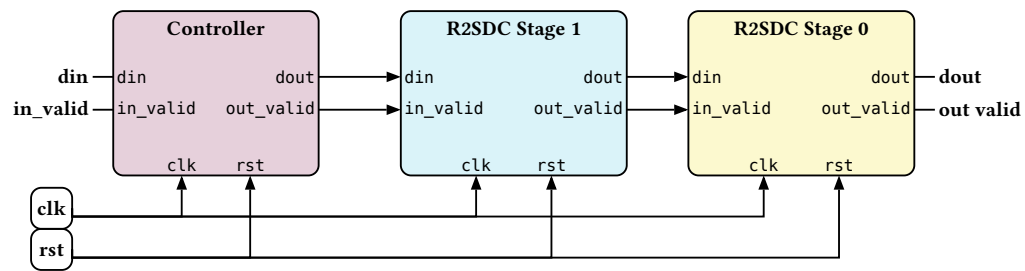


Figure 2.12: Schematic of R2SDC implementation

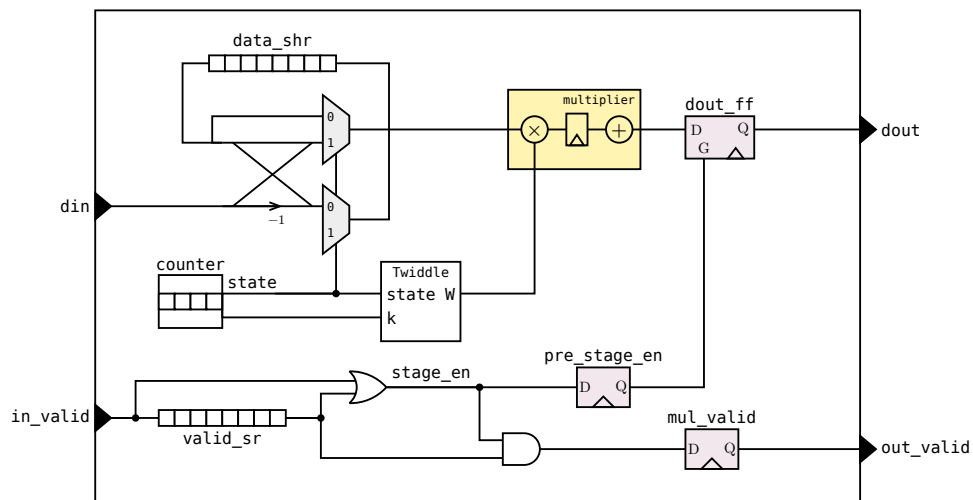


Figure 2.13: Inside a single R2SDC Stage. All modules with wedge is clocked

2.3.2 Simulation result

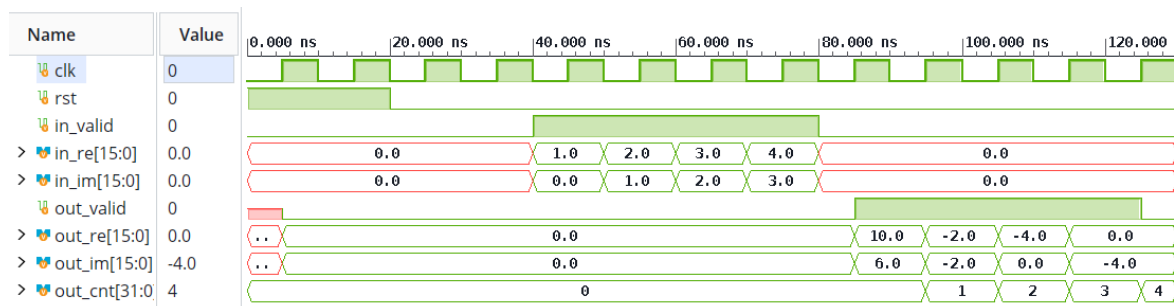


Figure 2.14: Simulation result for 4 point complex test signal

2.3.3 Implementation results

2.3.3.1. Memory, Nets & DSP

N	Total LUT	LUT/N	Logic LUT	LUT RAM	SRL	FF	FF/N	RAMB 36	RAMB 18	DSP Block	DSP Block/N
8	358	44.75	325	0	33	247	30.87	0	0	8	1.000
16	516	32.25	450	0	66	327	20.43	0	0	12	0.750
32	678	21.18	579	0	99	416	13	0	0	16	0.500
64	852	13.31	720	0	132	522	8.15	0	0	20	0.312
128	1072	8.37	874	0	198	661	5.16	0	0	24	0.187
256	1341	5.23	1011	0	330	865	3.37	0	2	28	0.109
512	1743	3.40	1149	0	594	1198	2.33	0	4	32	0.062
1024	2410	2.35	1288	0	1122	1788	1.74	0	6	36	0.035
2048	3642	1.77	1432	0	2210	2893	1.41	0	8	40	0.019
4096	5853	1.42	1563	0	4290	5035	1.22	2	8	44	0.010
8192	10213	1.24	1699	0	8514	9196	1.12	4	10	48	0.005

N	All Nets	Logical Net	All Nets/N	Logical Net/N	Power (mW) *	WNS (ns)	$F_{\max}$
8	3681	1323	460.12	165.37	145	1.406	116.36
16	5067	1824	316.68	114	154	1.549	118.32
32	6480	2340	202.5	73.12	163	1.218	113.86
64	7902	2874	123.46	44.90	172	0.951	110.50
128	8793	3499	68.69	27.33	185	0.920	110.13
256	10300	4265	40.23	16.66	197	0.735	107.93
512	12193	5293	23.81	10.33	211	1.219	113.88
1024	14553	6845	14.21	6.68	232	0.693	107.44
2048	16146	9478	7.88	4.62	263	0.657	107.03
4096	21727	14150	5.30	3.45	315	0.715	107.70
8192	31227	22993	3.81	2.80	393	0.445	104.65